

Generative AI & Agentic AI

From Large Language Models to Autonomous, Tool-Using Agents

This guide covers two closely related waves of AI development: generative AI, systems that produce new content — text, images, code, audio — rather than just classifying or predicting; and agentic AI, systems built on top of generative models that can plan, use tools, and act autonomously toward a goal. Together they represent the current frontier of applied AI systems.

1. What Makes AI "Generative"

Earlier machine learning systems were largely discriminative: given an input, predict a label or a number. Generative models instead learn the underlying distribution of the data well enough to produce new, plausible samples from it — a new sentence, a new image, a new piece of code. Large language models (LLMs) are the most prominent example: trained to predict the next token in a sequence of text, they end up learning enough about language, facts, and reasoning patterns to generate coherent, useful, long-form output.

2. Large Language Models

2.1 Pretraining

An LLM is first pretrained on a very large corpus of text using a simple objective: predict the next token given the tokens before it. This is done at massive scale — trillions of tokens and billions of parameters — using the transformer architecture described in the deep learning guide. Nothing in this stage is task-specific; the model is simply learning statistical patterns in language, which turns out to encode a surprising amount of world knowledge and reasoning capability as a byproduct.

2.2 Fine-Tuning and Alignment

A raw pretrained model is good at continuing text but not naturally good at following instructions or being helpful and safe. Supervised fine-tuning on curated instruction-response pairs teaches the model to follow instructions. Reinforcement learning from human feedback (RLHF), and newer variants like RLAIIF and direct preference optimization (DPO), further shape the model's behavior by training it to prefer outputs that human (or AI) evaluators rate more highly — for helpfulness, honesty, and safety.

2.3 Prompting and In-Context Learning

Because LLMs are trained on such broad data, they can perform many tasks without any additional training at all, simply from a well-written prompt — this is called zero-shot or few-shot prompting. Techniques like chain-of-thought prompting, where the model is asked to reason step by step before answering, measurably improve performance on tasks that require multi-step reasoning.

```
prompt = """
You are a support assistant. Classify the ticket below as
BILLING, TECHNICAL, or GENERAL, then explain your reasoning
in one sentence.

Ticket: "I was charged twice for my subscription this month."
"""

response = client.messages.create(
    model="claude-sonnet-5",
    messages=[{"role": "user", "content": prompt}]
```

)

Figure 2.1 — A simple zero-shot classification prompt

3. Retrieval-Augmented Generation (RAG)

LLMs have two structural limitations: their knowledge is frozen at training time, and they can state incorrect information confidently (hallucination). Retrieval-augmented generation addresses both by retrieving relevant documents from an external knowledge source at query time and inserting them into the model's context before it generates an answer, grounding the response in real, current, and verifiable source material.

- Documents are split into chunks and converted into vector embeddings.
- Embeddings are stored in a vector database (e.g. Pinecone, Weaviate, pgvector).
- At query time, the user's question is embedded and used to retrieve the most similar chunks.
- Retrieved chunks are inserted into the prompt as context before the model generates a response.

```
query_embedding = embed(user_question)
relevant_chunks = vector_db.search(query_embedding, top_k=5)

prompt = f"""Answer using only the context below.
Context: {relevant_chunks}
Question: {user_question}"""
answer = llm.generate(prompt)
```

Figure 3.1 — The core RAG retrieval-then-generate pattern

4. Beyond Text: Multimodal and Diffusion Models

Generative AI extends well beyond text. Diffusion models, used by systems like Stable Diffusion and Midjourney, generate images by starting from random noise and iteratively denoising it, guided by a text prompt, until a coherent image emerges. Multimodal models extend the transformer architecture to accept and produce combinations of text, images, and audio within a single unified model, rather than chaining separate systems together.

5. What Makes AI "Agentic"

An agent, in this context, is a system that uses an LLM not just to generate a single response, but to decide what to do next — repeatedly — in pursuit of a goal, often taking actions in the outside world through tools. The core loop is straightforward: the model observes the current state, decides on an action (which might be calling a tool, asking a clarifying question, or finishing the task), observes the result, and repeats until the goal is achieved or a stopping condition is hit.

- Reasoning — breaking a goal down into a sequence of steps.
- Tool use — calling external functions: search, code execution, APIs, databases.
- Memory — retaining relevant context across a long-running task.
- Planning and self-correction — adjusting the plan when a step fails or new information appears.

6. Core Agentic Patterns

6.1 The ReAct Loop

ReAct (Reason + Act) interleaves reasoning steps with tool calls: the model writes out its reasoning, decides on an action, observes the result of that action, and continues. This loop is the foundation underneath most modern agent frameworks and tool-calling APIs.

```
while not task_complete:
    thought = llm.reason(state)
    action = llm.choose_action(thought, available_tools)
    result = execute(action)
    state = update_state(state, action, result)
    task_complete = llm.check_done(state)
```

Figure 6.1 — A simplified ReAct-style agent loop

6.2 Tool Use and Function Calling

Modern LLM APIs support structured tool calling: the model is given a list of available tools with typed parameters, and instead of just producing text, it can output a request to call one of those tools with specific arguments. The calling application executes the tool and feeds the result back to the model, which then decides what to do next.

```
tools = [{
    "name": "get_weather",
    "description": "Get current weather for a city",
    "input_schema": {
        "type": "object",
        "properties": {"city": {"type": "string"}},
        "required": ["city"]
    }
}]
```

```
    }}  
    response = client.messages.create(  
        model="claude-sonnet-5",  
        tools=tools,  
        messages=[{"role": "user", "content": "Should I bring an umbrella in Chennai?"}]  
    )
```

Figure 6.2 — Defining a tool for an LLM to call

6.3 Multi-Agent Systems

For complex tasks, a single agent with a single set of instructions can become unwieldy. Multi-agent architectures split responsibility across specialized agents — a planner, a researcher, a coder, a reviewer — that communicate with each other, either through a shared orchestrator or by passing messages directly. This mirrors how human teams divide complex work, and it can make each individual agent's prompt and toolset simpler and more reliable.

7. Building and Evaluating Agents

Consideration	Why It Matters
Tool design	Clear, narrow tool definitions reduce misuse and ambiguity
Guardrails	Confirm before irreversible actions (sending email, deleting data)
Observability	Log every reasoning step and tool call for debugging
Cost and latency	Each reasoning step and tool call adds time and token cost
Evaluation	Test against realistic tasks, not just isolated prompts
Error recovery	Agents need a defined path when a tool call fails

Table 7.1 — Practical considerations when building agentic systems

7.1 Frameworks

A number of frameworks have emerged to reduce the boilerplate of building agents: LangChain and LangGraph for composable chains and stateful agent graphs, the Claude Agent SDK and OpenAI's Assistants API for provider-native agent loops, and CrewAI or AutoGen for multi-agent orchestration. Most of these frameworks converge on the same underlying pattern — a loop of reasoning, tool calls, and state updates — differing mainly in how much structure and abstraction they add on top.

8. Where Generative and Agentic AI Are Applied

- Coding assistants that read a codebase, write patches, and run tests autonomously.
- Customer support agents that look up account data and resolve tickets end-to-end.
- Research assistants that search, read, and synthesize information across many sources.
- Content generation for marketing, design, and media production.
- Business process automation — form processing, data entry, report generation.
- Personal productivity agents that manage email, calendars, and task lists.

9. Risks and Open Challenges

- Hallucination — generative models can produce fluent but factually incorrect content.
- Reliability at scale — small per-step error rates compound over long agent trajectories.
- Security — prompt injection can hijack an agent's tool-calling behavior if inputs aren't handled carefully.

- Cost — long agentic loops with many tool calls can become expensive in tokens and latency.
- Evaluation difficulty — open-ended agent behavior is harder to test than a single-turn model output.

10. Closing Thoughts

Generative AI changed what models can produce; agentic AI is changing what models can do. The two build directly on the deep learning and machine learning foundations covered earlier — the same transformer architecture, the same training techniques — but layered with new capabilities: retrieval, tool use, planning, and multi-step autonomy. As these systems mature, the central engineering challenge is shifting from "can the model produce a good answer" to "can the system reliably take the right actions, in the right order, with appropriate safeguards, over a long and unpredictable task."